Experiment 3 – Cursor Based Row Processing in PL/pgSQL

Student Name: Sahil Goyal
UID: 24BDA70148
Branch: CSE
Section/Group: AIT-KRG-GP2
Semester: 4th
Subject Name: DBMS

Experiment

Experiment 7: Design and implementation of simple views, complex views, and materialized views in PostgreSQL and comparison of their query execution performance.

Aim

To design and implement a **materialized view** and analyze the **performance difference between simple views, complex views, and materialized views**.

Objective

- To understand the concept of **database views**
- To create **simple views and complex views**
- To implement **materialized views in PostgreSQL**
- To compare **query execution performance**
- To understand the benefits of **materialized views in enterprise systems**

Software Requirements

- Database: PostgreSQL
- Tool: iterm2

Practical / Experiment Steps

1. reate tables for **Employee, Department, and Payroll**.
2. Insert sample records into the tables.
3. Create a **Simple View** based on the Employee table.
4. Create a **Complex View** using JOIN operations.
5. Create a **Materialized View** for storing precomputed results.
6. Execute queries on each view.
7. Analyze execution performance using **EXPLAIN ANALYZE**.

Input / Output Details

Input

Tables used:

- Employee
- Department
- Payroll

Operations performed:

- Creation of views
- Execution of queries
- Performance comparison

Output

- Display records from **Simple View**
- Display records from **Complex View**

- Display records from **Materialized View**
- Show execution time comparison using **EXPLAIN ANALYZE**

SQL / PL/pgSQL Code

1.TABLE CREATION

```
CREATE TABLE Employee (
    EmpID INT PRIMARY KEY,
    EmpName VARCHAR(50),
    DeptID INT,
    Salary NUMERIC(10,2)
);

CREATE TABLE Department (
    DeptID INT PRIMARY KEY,
    DeptName VARCHAR(50)
);

CREATE TABLE Payroll (
    EmpID INT,
    PayDate DATE,
    PayAmount NUMERIC(10,2),
    PRIMARY KEY (EmpID, PayDate)
);
```

2.INSERT DATA

```
INSERT INTO Department VALUES (1,'IT');
INSERT INTO Department VALUES (2,'Finance');
INSERT INTO Department VALUES (3,'HR');

INSERT INTO Employee VALUES (101,'Alice',1,70000);
INSERT INTO Employee VALUES (102,'Bob',2,80000);
INSERT INTO Employee VALUES (103,'Charlie',1,65000);
INSERT INTO Employee VALUES (104,'David',3,60000);

INSERT INTO Payroll VALUES (101,'2026-02-01',7000);
INSERT INTO Payroll VALUES (102,'2026-02-01',8000);
INSERT INTO Payroll VALUES (103,'2026-02-01',6500);
INSERT INTO Payroll VALUES (104,'2026-02-01',6000);
```

3.SIMPLE VIEW

```
CREATE VIEW SimpleEmployeeView AS
SELECT EmpID, EmpName, Salary
FROM Employee;

SELECT * FROM SimpleEmployeeView;
```

4.COMPLEX VIEW

```
CREATE VIEW ComplexEmployeePayroll AS
SELECT e.EmpID, e.EmpName, d.DeptName,
       SUM(p.PayAmount) AS TotalPaid
FROM Employee e
JOIN Department d ON e.DeptID = d.DeptID
JOIN Payroll p ON e.EmpID = p.EmpID
GROUP BY e.EmpID, e.EmpName, d.DeptName;

SELECT * FROM ComplexEmployeePayroll;
```

5.MATERALIZED VIEW

```
CREATE MATERIALIZED VIEW MaterializedEmployeePayroll AS
SELECT e.EmpID, e.EmpName, d.DeptName,
     SUM(p.PayAmount) AS TotalPaid
FROM Employee e
JOIN Department d ON e.DeptID = d.DeptID
JOIN Payroll p ON e.EmpID = p.EmpID
GROUP BY e.EmpID, e.EmpName, d.DeptName;

SELECT * FROM MaterializedEmployeePayroll;
```

6.PERFORMANCE ANALYSIS

```
EXPLAIN ANALYZE SELECT * FROM SimpleEmployeeView;
EXPLAIN ANALYZE SELECT * FROM ComplexEmployeePayroll;
EXPLAIN ANALYZE SELECT * FROM MaterializedEmployeePayroll;
```

Screenshots

- **Screenshot 1:** Table creation and data insertion

```
sahil=# CREATE TABLE Employee (
sahil(#     EmpID INT PRIMARY KEY,
sahil(#     EmpName VARCHAR(50),
sahil(#     DeptID INT,
sahil(#     Salary NUMERIC(10,2)
sahil(# );
CREATE TABLE
sahil=#
sahil=# CREATE TABLE Department (
sahil(#     DeptID INT PRIMARY KEY,
sahil(#     DeptName VARCHAR(50)
sahil(# );
CREATE TABLE
sahil=#
sahil=# CREATE TABLE Payroll (
sahil(#     EmpID INT,
sahil(#     PayDate DATE,
sahil(#     PayAmount NUMERIC(10,2),
sahil(#     PRIMARY KEY (EmpID, PayDate)
sahil(# );
CREATE TABLE
sahil=# INSERT INTO Department VALUES (1,'IT');
INSERT 0 1
sahil=# INSERT INTO Department VALUES (2,'Finance');
INSERT 0 1
sahil=# INSERT INTO Department VALUES (3,'HR');
INSERT 0 1
sahil=#
sahil=# INSERT INTO Employee VALUES (101,'Alice',1,70000);
INSERT 0 1
sahil=# INSERT INTO Employee VALUES (102,'Bob',2,80000);
INSERT 0 1
sahil=# INSERT INTO Employee VALUES (103,'Charlie',1,65000);
INSERT 0 1
sahil=# INSERT INTO Employee VALUES (104,'David',3,60000);
INSERT 0 1
sahil=#
sahil=# INSERT INTO Payroll VALUES (101,'2026-02-01',7000);
INSERT 0 1
sahil=# INSERT INTO Payroll VALUES (102,'2026-02-01',8000);
INSERT 0 1
sahil=# INSERT INTO Payroll VALUES (103,'2026-02-01',6500);
INSERT 0 1
sahil=# INSERT INTO Payroll VALUES (104,'2026-02-01',6000);
INSERT 0 1
```

- **Screenshot 2:** Output of Simple View

```
sahil=# CREATE VIEW SimpleEmployeeView AS
sahil-# SELECT EmpID, EmpName, Salary
sahil-# FROM Employee;
CREATE VIEW
sahil=#
sahil=# SELECT * FROM SimpleEmployeeView;
 empid | empname |  salary
-------+---------+----------
   101 | Alice   | 70000.00
   102 | Bob     | 80000.00
   103 | Charlie | 65000.00
   104 | David   | 60000.00
(4 rows)
```

- **Screenshot 3:** Output of Complex View

```
sahil=# CREATE VIEW ComplexEmployeePayroll AS
sahil-# SELECT e.EmpID, e.EmpName, d.DeptName,
sahil-#        SUM(p.PayAmount) AS TotalPaid
sahil-# FROM Employee e
sahil-# JOIN Department d ON e.DeptID = d.DeptID
sahil-# JOIN Payroll p ON e.EmpID = p.EmpID
sahil-# GROUP BY e.EmpID, e.EmpName, d.DeptName;
CREATE VIEW
sahil=#
sahil=# SELECT * FROM ComplexEmployeePayroll;
 empid | empname | deptname | totalpaid
-------+---------+----------+-----------
   102 | Bob     | Finance  |   8000.00
   104 | David   | HR       |   6000.00
   101 | Alice   | IT       |   7000.00
   103 | Charlie | IT       |   6500.00
(4 rows)
```

- **Screenshot 4:** Output of Materialized View

```
sahil=# CREATE MATERIALIZED VIEW MaterializedEmployeePayroll AS
sahil-# SELECT e.EmpID, e.EmpName, d.DeptName,
sahil-#        SUM(p.PayAmount) AS TotalPaid
sahil-# FROM Employee e
sahil-# JOIN Department d ON e.DeptID = d.DeptID
sahil-# JOIN Payroll p ON e.EmpID = p.EmpID
sahil-# GROUP BY e.EmpID, e.EmpName, d.DeptName;
SELECT 4
sahil=#
sahil=# SELECT * FROM MaterializedEmployeePayroll;
 empid | empname | deptname | totalpaid
-------+---------+----------+-----------
   102 | Bob     | Finance  |   8000.00
   104 | David   | HR       |   6000.00
   101 | Alice   | IT       |   7000.00
   103 | Charlie | IT       |   6500.00
(4 rows)
```

- **Screenshot 5:** Performance comparison using EXPLAIN ANALYZE

```
sahil=# EXPLAIN ANALYZE SELECT * FROM SimpleEmployeeView;
                                                 QUERY PLAN
-----------------------------------------------------------------------------------------------------
 Seq Scan on employee  (cost=0.00..14.80 rows=480 width=138) (actual time=0.022..0.026 rows=4.00 loops=1)
   Buffers: shared hit=1
 Planning Time: 0.103 ms
 Execution Time: 0.054 ms
(4 rows)

sahil=#
sahil=# EXPLAIN ANALYZE SELECT * FROM ComplexEmployeePayroll;
                                                          QUERY PLAN
-----------------------------------------------------------------------------------------------------------------------------
 HashAggregate  (cost=88.73..108.36 rows=1570 width=272) (actual time=0.074..0.082 rows=4.00 loops=1)
   Group Key: e.empid, d.deptname
   Batches: 1  Memory Usage: 65kB
   Buffers: shared hit=3
   ->  Hash Join  (cost=42.95..76.96 rows=1570 width=256) (actual time=0.057..0.063 rows=4.00 loops=1)
         Hash Cond: (e.deptid = d.deptid)
         Buffers: shared hit=3
         ->  Hash Join  (cost=20.80..50.65 rows=1570 width=142) (actual time=0.025..0.028 rows=4.00 loops=1)
               Hash Cond: (p.empid = e.empid)
               Buffers: shared hit=2
               ->  Seq Scan on payroll p  (cost=0.00..25.70 rows=1570 width=20) (actual time=0.005..0.006 rows=4.00 loops=1)
                     Buffers: shared hit=1
               ->  Hash  (cost=14.80..14.80 rows=480 width=126) (actual time=0.009..0.009 rows=4.00 loops=1)
                     Buckets: 1024  Batches: 1  Memory Usage: 9kB
                     Buffers: shared hit=1
                     ->  Seq Scan on employee e  (cost=0.00..14.80 rows=480 width=126) (actual time=0.005..0.006 rows=4.00 loops=1)
                           Buffers: shared hit=1
         ->  Hash  (cost=15.40..15.40 rows=540 width=122) (actual time=0.022..0.023 rows=3.00 loops=1)
               Buckets: 1024  Batches: 1  Memory Usage: 9kB
               Buffers: shared hit=1
               ->  Seq Scan on department d  (cost=0.00..15.40 rows=540 width=122) (actual time=0.016..0.017 rows=3.00 loops=1)
                     Buffers: shared hit=1
 Planning Time: 0.265 ms
 Execution Time: 0.199 ms
(24 rows)

sahil=#
sahil=# EXPLAIN ANALYZE SELECT * FROM MaterializedEmployeePayroll;
                                                       QUERY PLAN
-----------------------------------------------------------------------------------------------------------------------
 Seq Scan on materializedemployeepayroll  (cost=0.00..12.70 rows=270 width=272) (actual time=0.016..0.018 rows=4.00 loops=1)
   Buffers: shared hit=1
 Planning Time: 0.065 ms
 Execution Time: 0.036 ms
(4 rows)
```

Learning Outcome

- Learned the concept and purpose of **database views** in PostgreSQL.
- Understood the difference between **simple views, complex views, and materialized views**.
- Learned how to create and query **simple views based on a single table**.
- Learned how to create **complex views using joins and aggregation functions**.
- Understood how **materialized views store precomputed results** to improve query performance.
- Gained knowledge of using **EXPLAIN ANALYZE** to measure and compare query execution time.
- Developed an understanding of how **materialized views improve performance in data-intensive enterprise applications**.